CS2201 Computer Organization & Architecture

L30: Input/Output Organization



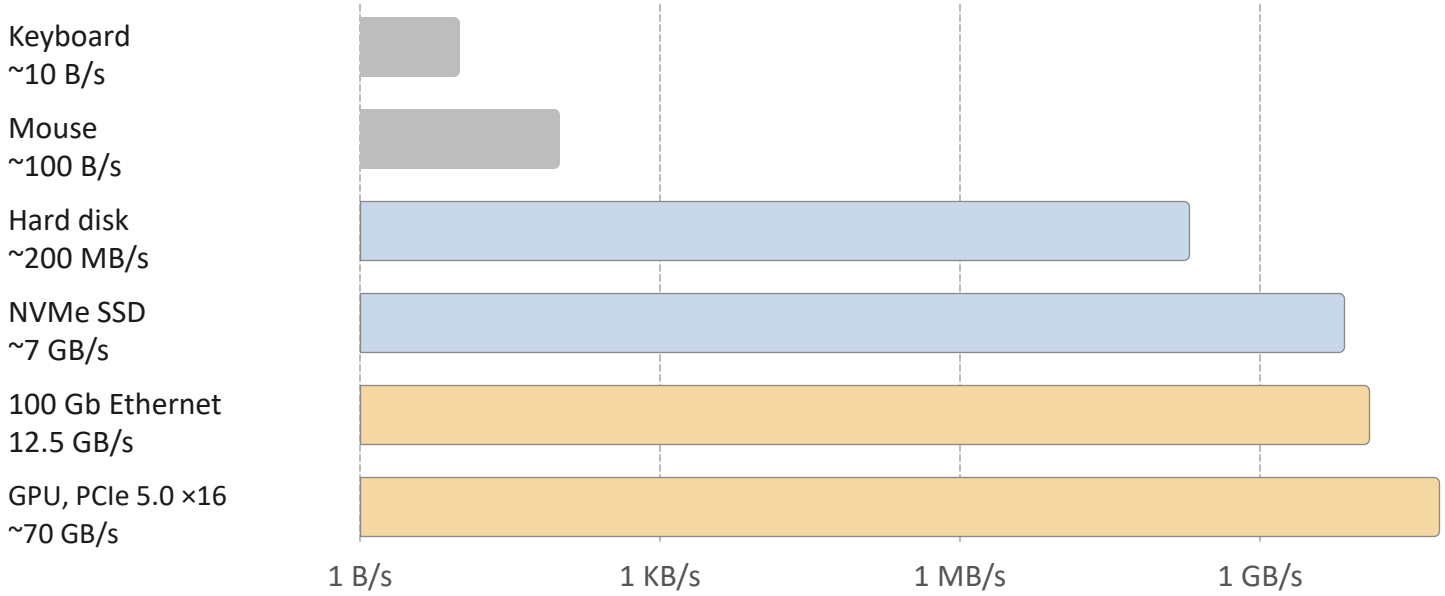
John Jose

Professor

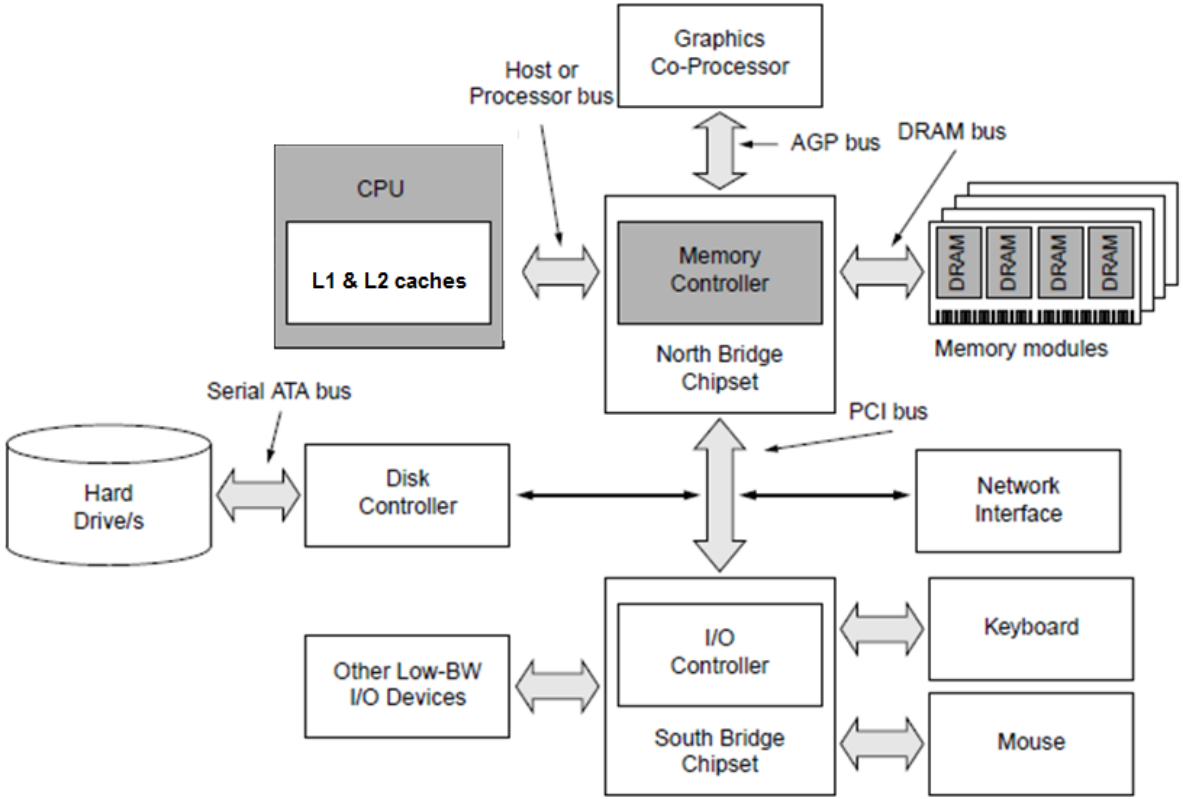
Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

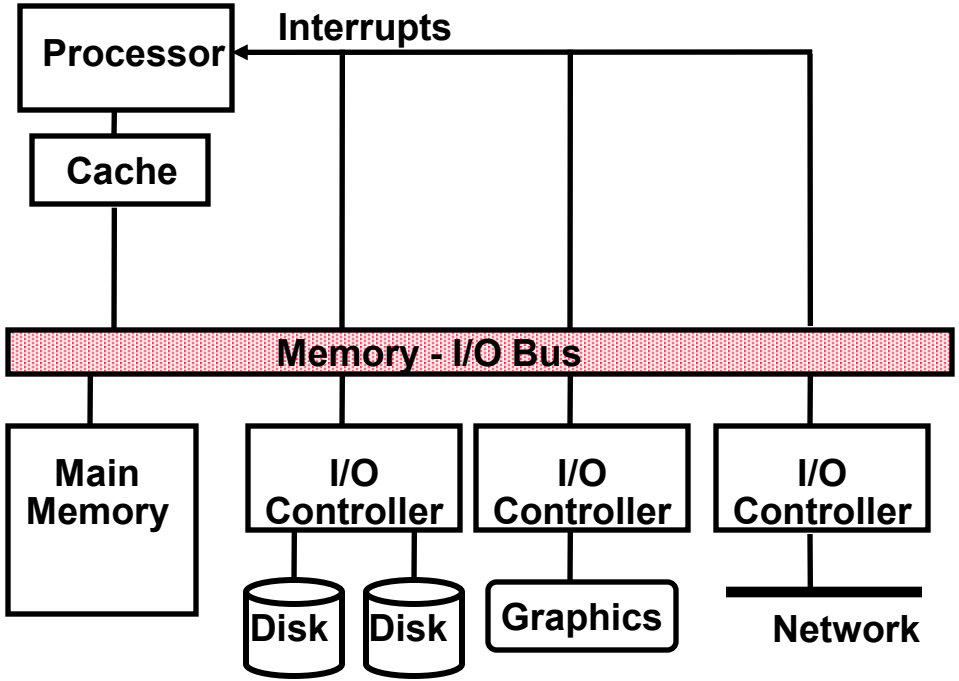
The I/O problem



Role of I/O controller



Typical I/O Subsystem

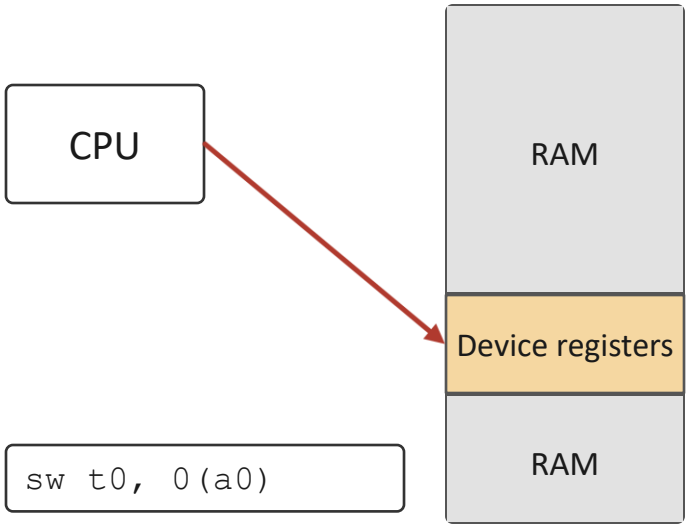


I/O Mapping

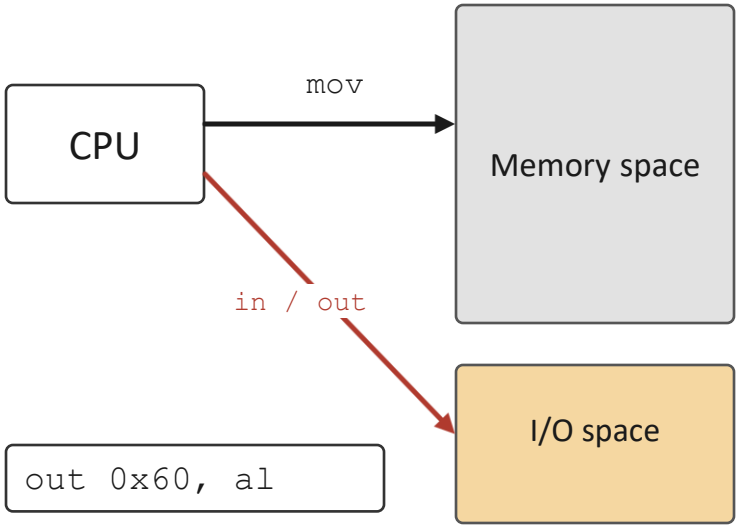
- **Memory mapped I/O**
  - Devices and memory share an address space
  - I/O looks just like memory read/write
  - No special commands for I/O
  - Large selection of memory access commands available
- **Isolated I/O (I/O mapped I/O)**
  - Separate address spaces
  - Need I/O or memory select lines
  - Special commands for I/O; Limited set

Memory-mapped vs Isolated I/O

Memory-mapped I/O

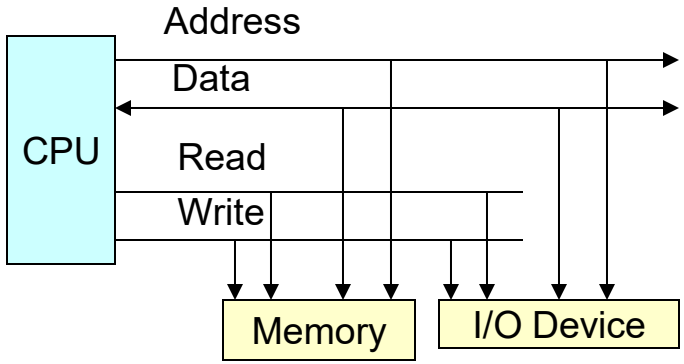


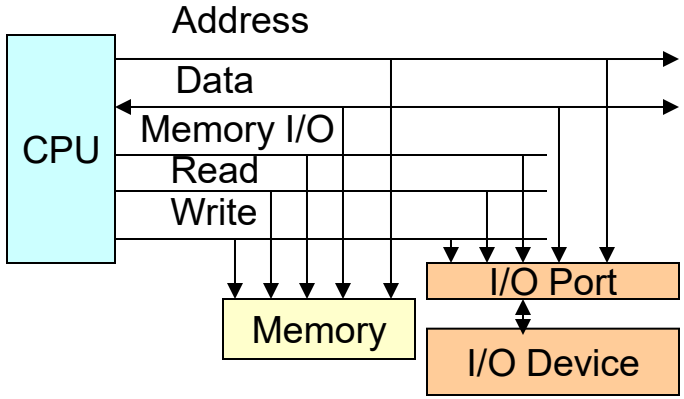
Isolated (port-mapped) I/O



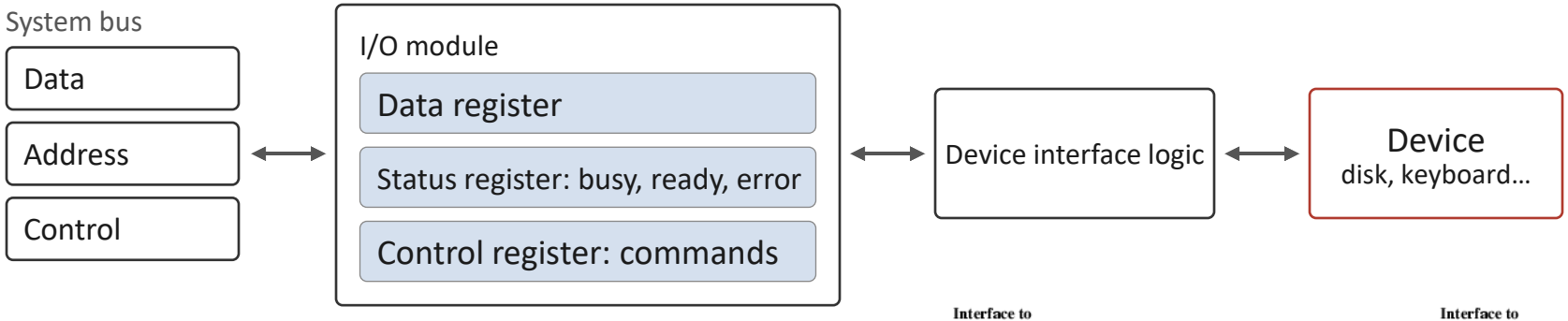
I/O Mapping

- CPU needs to talk to I/O
- **Memory mapped I/O**
  - Devices mapped to reserved memory locations - like RAM
  - Uses load/store instructions just like accesses to memory
- **I/O mapped I/O**
  - Special bus line
  - Special instructions

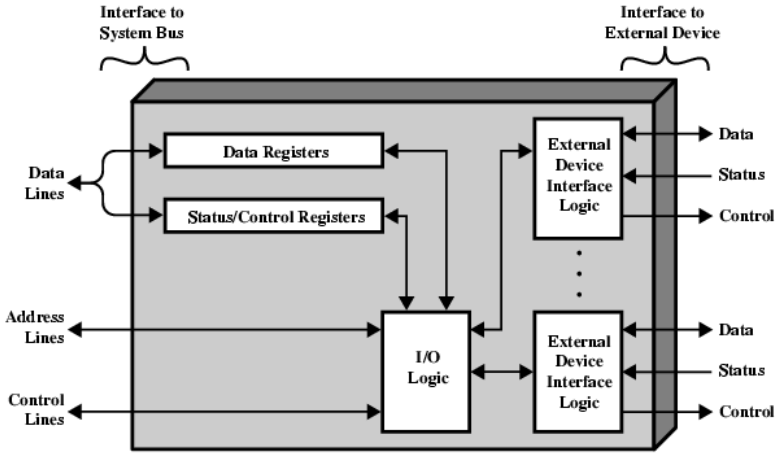




I/O module

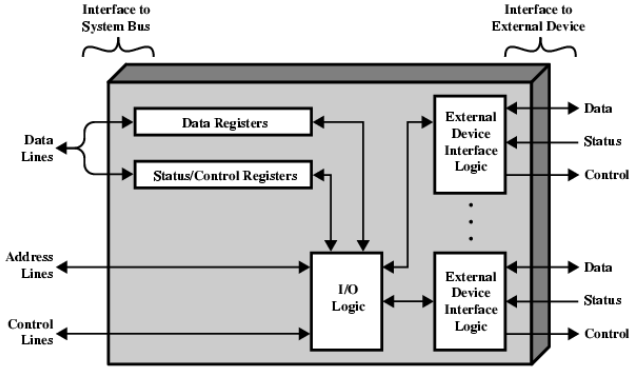


- Control & Timing
- Processor Communication
- Device Communication
- Data Buffering
- Error Detection and Correction



I/O Controller and Device Interfacing

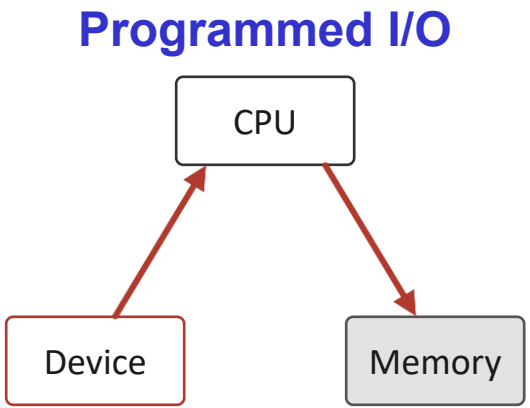
- I/O module interface I/O to CPU and Memory
- **I/O controller ←→ I/O devices ports**
  - Transfers data to/from device
  - Synchronizes operations
- **Status/ control registers**: device status, errors
- **Data registers**
  - Write: CPU/RAM data ➔ device [Transmit]
  - Read: CPU ➔ device [Receive]



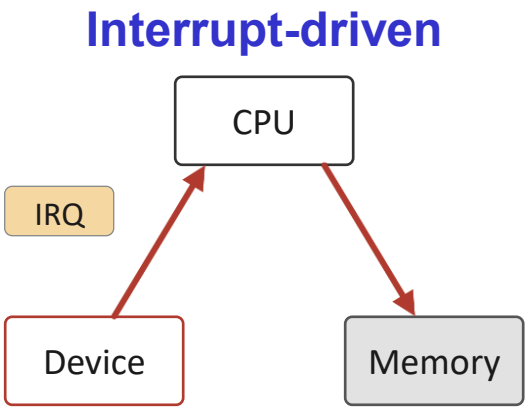
I/O Controller Module

I/O Data Transfer techniques

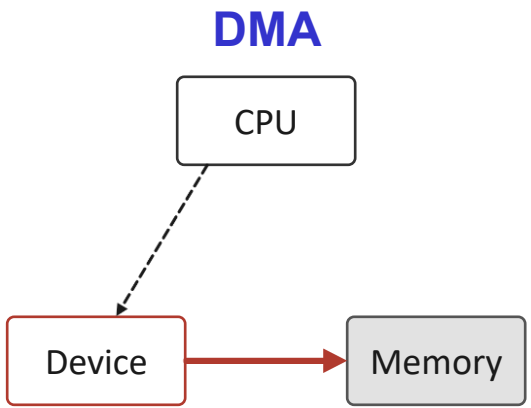
- **Programmed I/O**
- **Interrupt-Driven I/O**
- **Direct Memory Access (DMA)**



CPU polls, then copies word

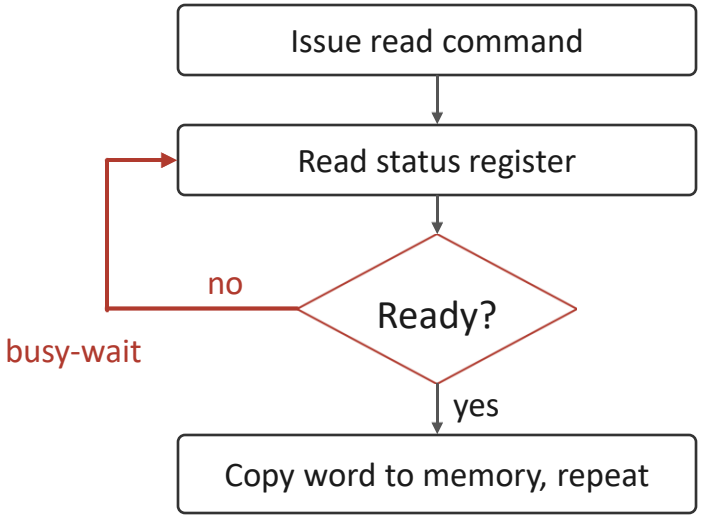


Device interrupts; CPU copies word



CPU only sets up; a DMA engine copies

Programmed I/O



- CPU periodically check I/O status (polling)
  - If device ready, do operation
- CPU has direct control over I/O
  - Sensing status
  - Read/write commands
  - Transferring data
- CPU waits for I/O completion
- Wastes CPU time due to polling
- I/O module does not inform CPU directly
- I/O module does not interrupt CPU

Cost of Polling

Assume a 2 GHz CPU and 400 cycles per poll

Mouse, polled 30 times/s

30 × 400 = 12,000 cycles/s

0.0006%

of the CPU

100 MB/s device, 16 B per poll

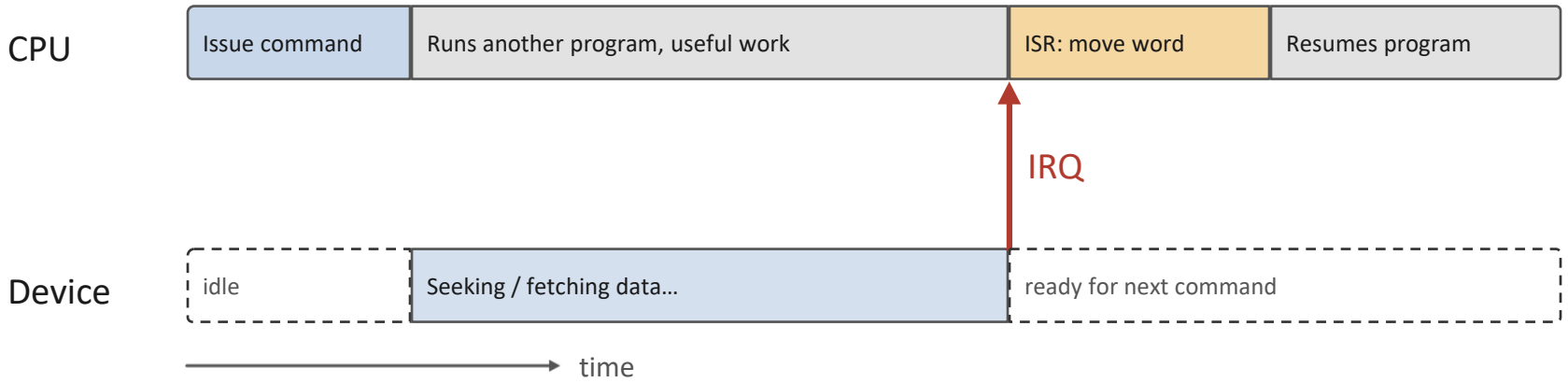
6.25 M polls/s × 400 = 2.5 G cycles/s

125%

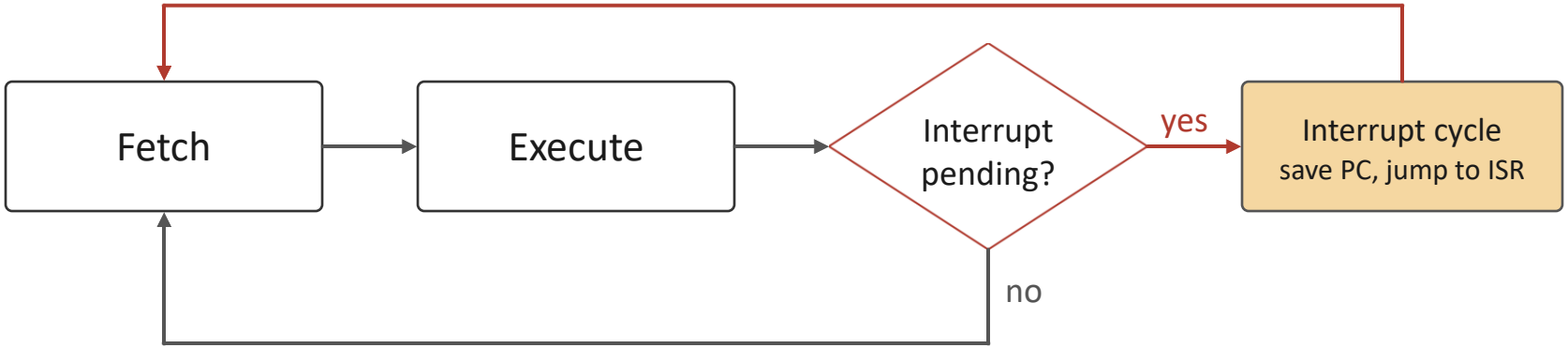
of the CPU: impossible

Interrupt-driven I/O

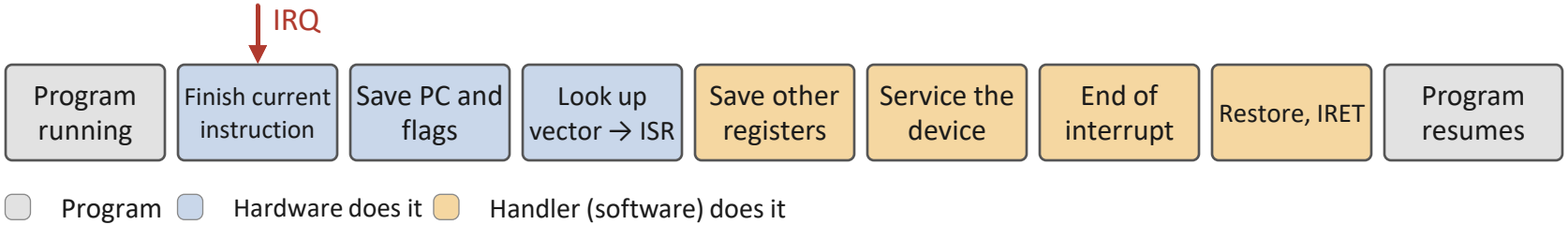
- An **interrupt** in I/O (input/output) processing is a signal sent to the CPU by an I/O device/ controller to notify it that an operation has finished, data is ready to be read, or an error has occurred.
- CPU respond back if an interrupt is raised.



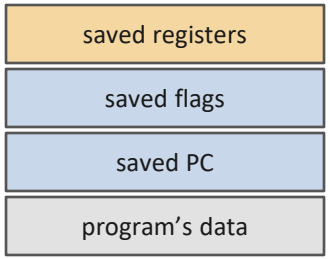
Instruction Cycle with Interrupts



Interrupt Processing



Stack

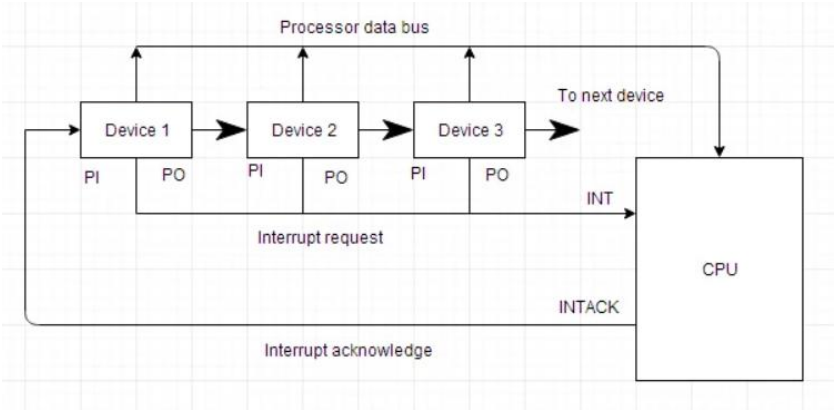


Challenges in Interrupt Driven

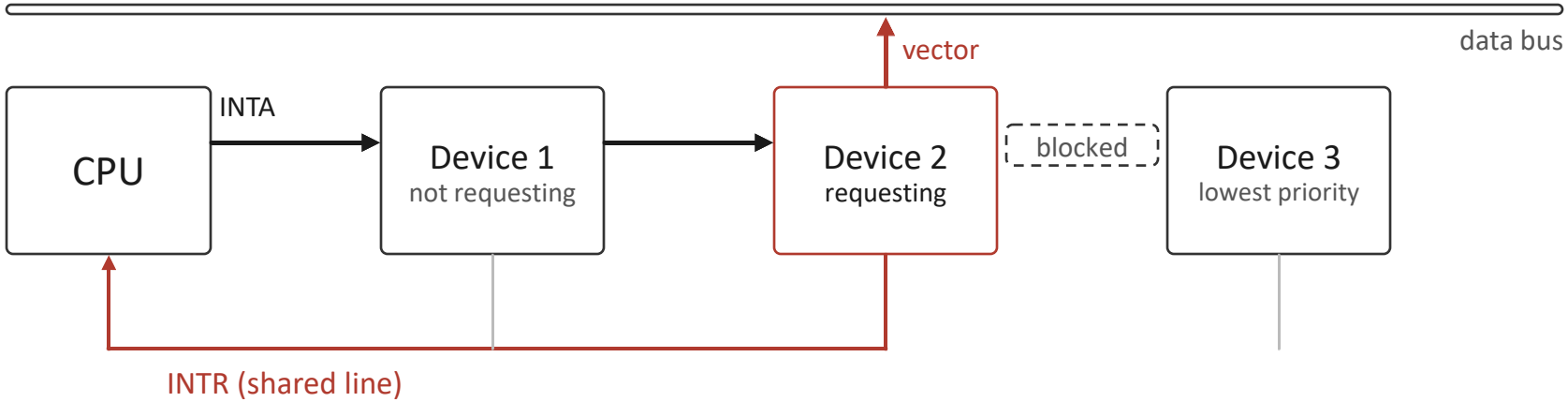
- How do you identify the module issuing the interrupt?
  - Need a way to identify the device generating the interrupt
  - Vectored vs Non-Vectored Interrupts
  - Interrupt Vector → ISR address
- How do you deal with multiple interrupts?
  - Can have different urgencies (so need a way to prioritize them)
  - Maskable vs Non-Maskable Interrupts

Identifying Interrupting Module

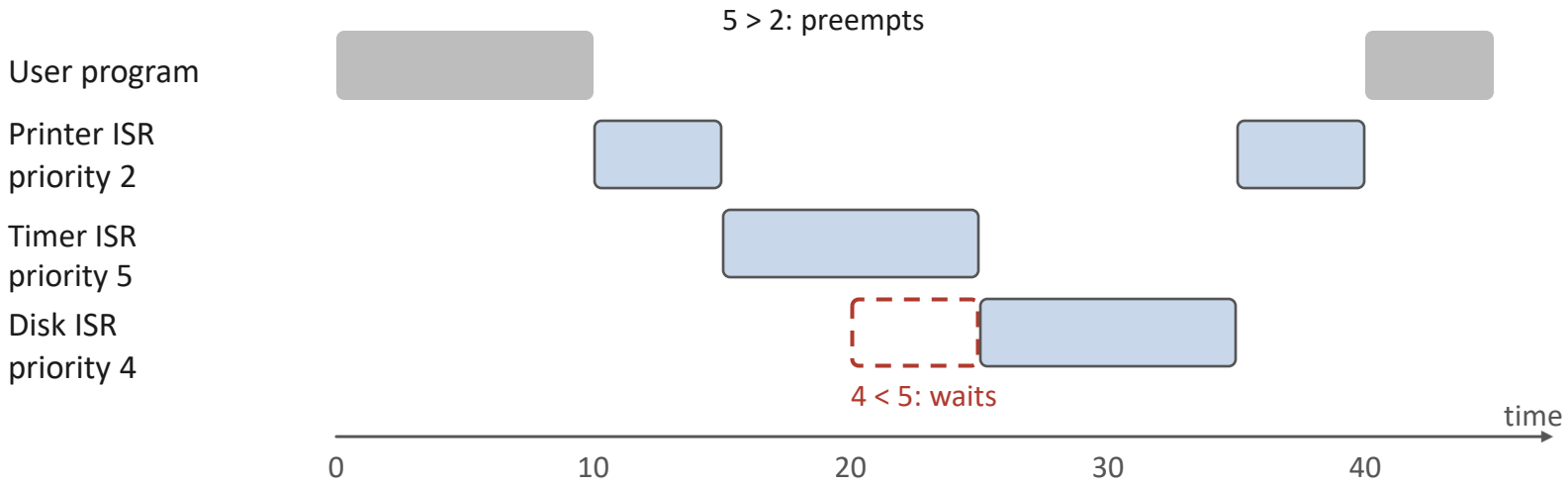
- CPU asks each module in turn (Slow) Daisy Chain or Hardware poll
  - Interrupt Acknowledge sent down a chain
  - Module responsible places vector on bus
  - CPU uses vector to identify handler routine



Identifying the interrupting device



Nested interrupts



Cost of interrupt-driven I/O

Same 100 MB/s device, 16 B per interrupt, 500 cycles per interrupt

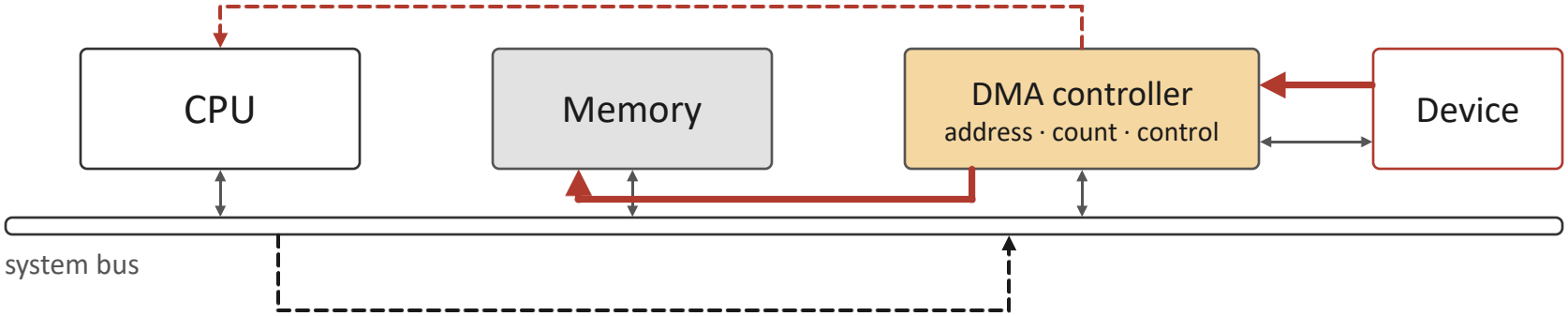
`6.25 M interrupts/s × 500 = 3.1 G cycles/s`

156%

of a 2 GHz CPU

→ DMA

Direct Memory Access (DMA)

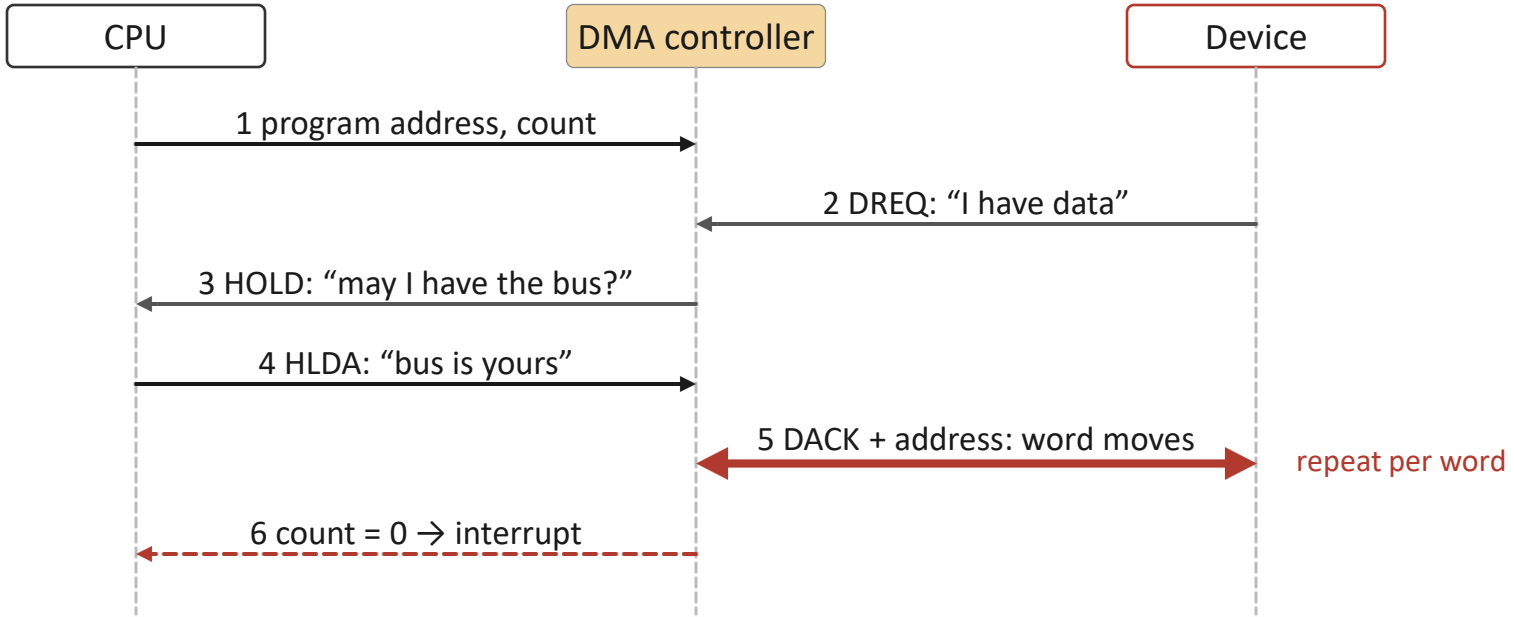


1 CPU programs the controller

2 Controller moves the block, device → memory

3 One interrupt at the end

DMA transfer sequence



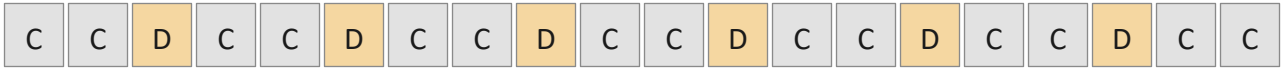
DMA modes

Burst



CPU locked out until the block is done

Cycle stealing



CPU slowed, never stopped

Transparent



Uses only idle bus cycles

C CPU owns the bus D DMA owns the bus

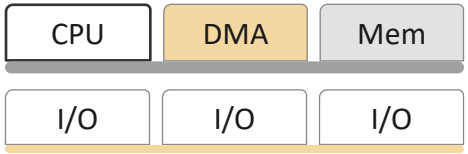
DMA configurations

Single bus, detached



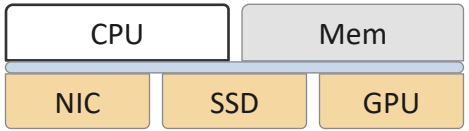
Each word crosses the bus twice

Integrated / I/O bus



Each word crosses the system bus once

Bus-master devices



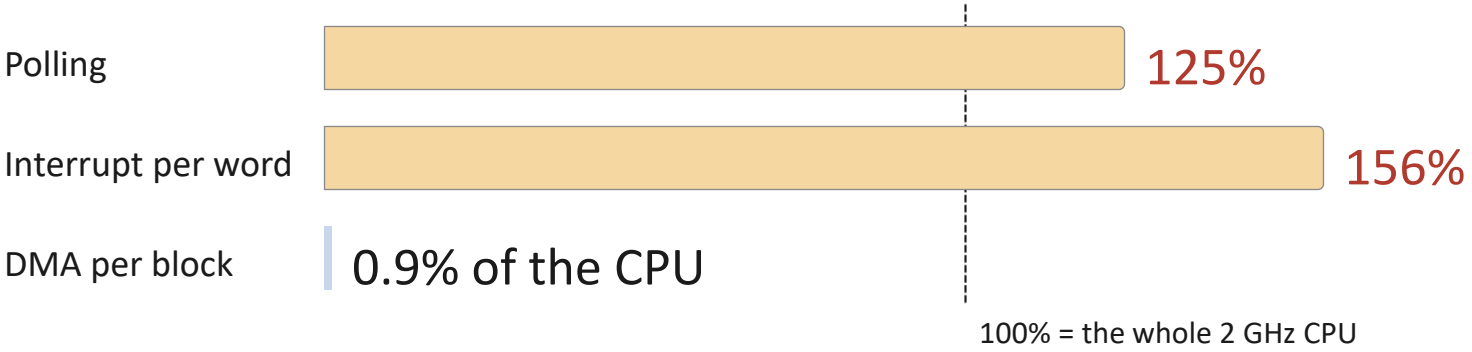
Every device is its own DMA engine, the PCIe norm

Shaded = contains a DMA engine

Cost of DMA

8 KB blocks · setup 1,000 cycles + one 500-cycle interrupt per block

```
100 MB/s ÷ 8 KB = 12,207 blocks/s × 1,500 = 18.3 M cycles/s
```



Summary

	Programmed I/O	Interrupt-driven	DMA
Moves data	CPU	CPU	DMA engine
CPU waits?	Yes, spins	No	No
Overhead per	Poll, plus every word	Word	Block
Good for	Simple MCUs; very fast, always-ready devices	Slow, sporadic devices	Bulk transfers: disk, network



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/